

ADVANCED SQL 3: TRIGGERS AND ASSERTIONS



Objective

The objective of this session is to enforce business rules automatically at the database level — using triggers to react to data changes, and using assertions (and PostgreSQL’s practical alternatives to them) to guarantee invariants that span rows and tables — using the dvdrental database from Lab 1.

Instructions

- Open the handout/lab manual in front of a computer in the lab during the session, with the dvdrental database already restored and available.
- Practice each new command by completing the examples and exercises.
- Turn in the answers for all the exercise problems as your lab report.
- When answering problems, indicate the commands you entered and the output displayed.

Contents

Introduction to Triggers	4
What Is a Trigger?	4
Trigger Timing and Level	4
Writing a Trigger Function	4
The Special NEW and OLD Records	4
TG_OP and Other Trigger Variables	5
RETURN Value Rules for Row-Level Triggers	5
CREATE TRIGGER Syntax	5
Binding a Trigger Function to a Table	5
The WHEN Clause	6
BEFORE Triggers: Validating and Modifying Data	6
Blocking Invalid Data	6
Auto-Populating a Column	7
AFTER Triggers: Auditing and Side Effects	7
A Simple Audit Log	7
Row-Level vs. Statement-Level Triggers	8
Transition Tables (REFERENCING ... AS)	8
INSTEAD OF Triggers on Views	8
Managing Triggers	9
Trigger Firing Order	9
Enabling, Disabling, and Dropping Triggers	9
Inspecting Existing Triggers in dvdrental	9
Assertions	10
The SQL Standard CREATE ASSERTION (and Why PostgreSQL Doesn't Have It)	10
A Self-Contained Example: A Small Banking Schema	10
Simulating Row-Level Assertions with CHECK	11
Simulating Cross-Row Assertions with Triggers	11
Deferrable Constraint Triggers	11
Putting It All Together	12
Exercise	14

Introduction to Triggers

The functions and procedures from Lab 3 only run when something explicitly calls them. A trigger is different: it is PL/pgSQL code that PostgreSQL runs automatically whenever a specified event happens on a table — an INSERT, UPDATE, DELETE, or TRUNCATE — with no application code needing to call anything.

What Is a Trigger?

A trigger has two parts, created separately:

1. A trigger function — an ordinary PL/pgSQL function that returns TRIGGER instead of a normal type.
2. A CREATE TRIGGER statement — which binds that function to a table and an event, telling PostgreSQL when to call it.

Note: This two-step design is deliberate: the same trigger function can be attached to several different tables or events, instead of writing the logic once per table.

Trigger Timing and Level

Two independent choices control exactly when a trigger fires:

- Timing — BEFORE (runs before the change is applied, and can still modify or cancel it), AFTER (runs once the change is already committed to the table), or INSTEAD OF (replaces the change entirely — used only on views).
- Level — FOR EACH ROW (fires once per affected row) or FOR EACH STATEMENT (fires once per SQL statement, regardless of how many rows it touched).

Writing a Trigger Function

A trigger function looks like any other PL/pgSQL function, except it takes no parameters and always returns TRIGGER:

```
CREATE OR REPLACE FUNCTION trigger_function_name()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- logic, using NEW / OLD / TG_OP as needed
    RETURN NEW; -- or OLD, or NULL
END;
$$;
```

Code language: SQL (Structured Query Language) (sql)

The Special NEW and OLD Records

Inside a row-level trigger, PostgreSQL automatically provides two record variables:

- NEW — the row as it will look after an INSERT or UPDATE. Not available on DELETE.
- OLD — the row as it looked before an UPDATE or DELETE. Not available on INSERT.

Note: On UPDATE, both NEW and OLD are available together, which is what makes it possible to compare old and new values inside the same trigger.

TG_OP and Other Trigger Variables

TG_OP tells the trigger which operation fired it, so one function can handle several events:

```
BEGIN
  IF TG_OP = 'INSERT' THEN
    RAISE NOTICE 'Row inserted into %', TG_TABLE_NAME;
  ELSIF TG_OP = 'UPDATE' THEN
    RAISE NOTICE 'Row updated in %', TG_TABLE_NAME;
  ELSIF TG_OP = 'DELETE' THEN
    RAISE NOTICE 'Row deleted from %', TG_TABLE_NAME;
  END IF;
  RETURN NEW;
END;
```

Code language: SQL (Structured Query Language) (sql)

Other useful automatic variables include TG_TABLE_NAME (the table the trigger fired on) and TG_WHEN (BEFORE, AFTER, or INSTEAD OF).

RETURN Value Rules for Row-Level Triggers

- A BEFORE row-level trigger should RETURN NEW to let the (possibly modified) row proceed, or RETURN NULL to silently skip the operation for that row.
- An AFTER row-level trigger's return value is ignored — the change has already happened — but it must still return something, conventionally NEW or OLD.
- A DELETE trigger has no NEW, so BEFORE DELETE returns OLD to let the deletion proceed, or NULL to cancel it.

CREATE TRIGGER Syntax

Binding a Trigger Function to a Table

```
CREATE TRIGGER trigger_name
  { BEFORE | AFTER | INSTEAD OF } { INSERT | UPDATE | DELETE }
  ON table_name
  FOR EACH { ROW | STATEMENT }
  [ WHEN (condition) ]
  EXECUTE FUNCTION trigger_function_name();
```

Code language: SQL (Structured Query Language) (sql)

A BEFORE INSERT trigger that rejects films with a non-positive rental_duration — logic a plain CHECK constraint could also express, shown first because it is the simplest possible trigger:

```
CREATE OR REPLACE FUNCTION check_rental_duration()
  RETURNS TRIGGER
  LANGUAGE plpgsql
  AS $$
  BEGIN
    IF NEW.rental_duration <= 0 THEN
      RAISE EXCEPTION 'rental_duration must be positive, got %', NEW.rental_duration;
    END IF;
```

```

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_check_rental_duration
    BEFORE INSERT OR UPDATE ON film
    FOR EACH ROW
    EXECUTE FUNCTION check_rental_duration();

```

Code language: SQL (Structured Query Language) (sql)

The WHEN Clause

WHEN lets PostgreSQL skip calling the function entirely when the condition is false, which is both clearer and faster than checking the same condition as the first line inside the function. This only fires when rental_rate actually changes, not on every UPDATE:

```

CREATE TRIGGER trg_log_rate_change
    AFTER UPDATE ON film
    FOR EACH ROW
    WHEN (OLD.rental_rate IS DISTINCT FROM NEW.rental_rate)
    EXECUTE FUNCTION log_rate_change();

```

Code language: SQL (Structured Query Language) (sql)

Note: IS DISTINCT FROM compares two values as NULL-safe — unlike <>, it treats NULL as a normal, comparable value instead of making the whole comparison unknown.

BEFORE Triggers: Validating and Modifying Data

Blocking Invalid Data

A BEFORE trigger can enforce a rule a plain FOREIGN KEY cannot express — for example, refusing to delete a category while it still has films assigned, exactly the behaviour a RESTRICT foreign key would give, written out explicitly as a trigger instead:

```

CREATE OR REPLACE FUNCTION prevent_category_delete_if_in_use()
    RETURNS TRIGGER
    LANGUAGE plpgsql
    AS $$
    BEGIN
        IF EXISTS (
            SELECT 1 FROM film_category WHERE category_id = OLD.category_id
        ) THEN
            RAISE EXCEPTION
                'cannot delete category % (%): % still has films assigned',
                OLD.category_id, OLD.name, OLD.name;
        END IF;
        RETURN OLD;
    END;
$$;

```

```
CREATE TRIGGER trg_protect_category
  BEFORE DELETE ON category
  FOR EACH ROW
  EXECUTE FUNCTION prevent_category_delete_if_in_use();
```

Code language: SQL (Structured Query Language) (sql)

Auto-Populating a Column

A BEFORE trigger can also modify NEW before it is written, which is how dvdrental already keeps its last_update columns current — every table in the schema has a trigger like this one bound to it:

```
CREATE OR REPLACE FUNCTION set_last_update()
  RETURNS TRIGGER
  LANGUAGE plpgsql
  AS $$
  BEGIN
    NEW.last_update := NOW();
    RETURN NEW;
  END;
  $$;

CREATE TRIGGER trg_set_last_update
  BEFORE UPDATE ON film
  FOR EACH ROW
  EXECUTE FUNCTION set_last_update();
```

Code language: SQL (Structured Query Language) (sql)

Note: Changing NEW inside a BEFORE trigger changes what actually gets written to the table — this only works in BEFORE triggers, since by AFTER the row is already committed.

AFTER Triggers: Auditing and Side Effects

AFTER triggers cannot change the row that just changed, but they are the natural place to record history, update a separate summary table, or trigger some other side effect.

A Simple Audit Log

First, a table to hold the history, then a trigger function that writes to it on every change to customer:

```
CREATE TABLE customer_audit (
  audit_id    SERIAL PRIMARY KEY,
  customer_id INT,
  action      VARCHAR(10),
  old_email   VARCHAR(50),
  new_email   VARCHAR(50),
  changed_at  TIMESTAMP DEFAULT NOW()
);

CREATE OR REPLACE FUNCTION log_customer_email_change()
  RETURNS TRIGGER
  LANGUAGE plpgsql
```

```

AS $$
BEGIN
    INSERT INTO customer_audit (customer_id, action, old_email, new_email)
    VALUES (NEW.customer_id, TG_OP, OLD.email, NEW.email);
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_log_customer_email
AFTER UPDATE ON customer
FOR EACH ROW
WHEN (OLD.email IS DISTINCT FROM NEW.email)
EXECUTE FUNCTION log_customer_email_change();

```

Code language: SQL (Structured Query Language) (sql)

Row-Level vs. Statement-Level Triggers

A FOR EACH ROW trigger fires once per affected row — ten rows updated means ten firings. A FOR EACH STATEMENT trigger fires exactly once per statement, no matter how many rows it touched, and it has no NEW or OLD (there could be many of each).

Transition Tables (REFERENCING ... AS)

A statement-level trigger can still see every row the statement affected, using a transition table — a temporary, query-able view of all the OLD or NEW rows from that one statement:

```

CREATE OR REPLACE FUNCTION log_bulk_price_change()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO customer_audit (action, changed_at)
    SELECT 'FILM_BATCH_UPDATE', NOW()
    FROM new_rows
    LIMIT 1;
    RETURN NULL;
END;
$$;

CREATE TRIGGER trg_log_bulk_price_change
AFTER UPDATE ON film
REFERENCING NEW TABLE AS new_rows
FOR EACH STATEMENT
EXECUTE FUNCTION log_bulk_price_change();

```

Code language: SQL (Structured Query Language) (sql)

Note: Transition tables are only available to AFTER triggers, and only the table's own INSERT/UPDATE/DELETE rows for that statement — not related tables.

INSTEAD OF Triggers on Views

A view built from a JOIN is usually not directly updatable. An INSTEAD OF trigger replaces an INSERT, UPDATE, or DELETE against the view with whatever logic you write — typically writing to the underlying base tables instead:

```
CREATE VIEW customer_city AS
SELECT c.customer_id, c.first_name, c.last_name, ci.city
FROM customer c
JOIN address a ON a.address_id = c.address_id
JOIN city ci    ON ci.city_id = a.city_id;

CREATE OR REPLACE FUNCTION customer_city_update()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE customer
    SET first_name = NEW.first_name, last_name = NEW.last_name
    WHERE customer_id = OLD.customer_id;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_customer_city_update
    INSTEAD OF UPDATE ON customer_city
    FOR EACH ROW
    EXECUTE FUNCTION customer_city_update();

UPDATE customer_city SET first_name = 'MARY-JANE' WHERE customer_id = 1;
```

Code language: SQL (Structured Query Language) (sql)

Note: INSTEAD OF triggers are always FOR EACH ROW — PostgreSQL does not allow FOR EACH STATEMENT on a view.

Managing Triggers

Trigger Firing Order

When more than one trigger of the same timing and event is attached to a table, PostgreSQL fires them in alphabetical order by trigger name — which is why naming triggers with a consistent prefix like `trg_` keeps their order predictable.

Enabling, Disabling, and Dropping Triggers

```
ALTER TABLE film DISABLE TRIGGER trg_check_rental_duration;
ALTER TABLE film ENABLE TRIGGER trg_check_rental_duration;
DROP TRIGGER IF EXISTS trg_check_rental_duration ON film;
```

Code language: SQL (Structured Query Language) (sql)

Inspecting Existing Triggers in dvdrental

dvdrental already relies on triggers to keep its own `last_update` columns current on every table. You can list every user-defined trigger already in the database:

```
SELECT tname AS trigger_name, tgrelid::regclass AS table_name
FROM pg_trigger
WHERE NOT tgisinternal
ORDER BY table_name;
```

Code language: SQL (Structured Query Language) (sql)

Assertions

Every constraint covered so far — PRIMARY KEY, FOREIGN KEY, CHECK, UNIQUE — is attached to one table and, in the case of CHECK, evaluated against one row at a time. An assertion is different: it is a standalone, named invariant that must hold across the database as a whole, checked automatically after every statement that could affect it, regardless of which table changed.

The SQL Standard CREATE ASSERTION (and Why PostgreSQL Doesn't Have It)

The SQL standard defines assertions with a simple, table-independent syntax:

```
-- Standard SQL syntax – NOT supported by PostgreSQL
CREATE ASSERTION assertion_name
CHECK (condition_that_must_always_be_true);
```

Code language: SQL (Structured Query Language) (sql)

Note: No mainstream database — PostgreSQL included — implements CREATE ASSERTION, mainly because efficiently re-checking an arbitrary condition against the whole database after every write is extremely expensive. In practice, the same guarantee is built from tools already covered in this course: CHECK constraints, triggers, and a PostgreSQL-specific feature called a constraint trigger.

A Self-Contained Example: A Small Banking Schema

dvdrntal has no natural example of a value that must stay constant across many rows, which is exactly the kind of rule an assertion exists for. The classic textbook case is a bank: money moved out of one account must be added to another, so the total balance across every account should never change because of a transfer. This section builds that schema from scratch, with real seed data, and uses it for the rest of the section:

```
CREATE TABLE bank_account (
    account_id SERIAL PRIMARY KEY,
    owner_name VARCHAR(100) NOT NULL,
    balance NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (balance >= 0)
);

INSERT INTO bank_account (owner_name, balance) VALUES
    ('Alice Johnson', 500.00),
    ('Bob Martinez', 250.00),
    ('Carla Diaz', 1000.00);

CREATE TABLE bank_transaction (
    transaction_id SERIAL PRIMARY KEY,
    from_account_id INT NOT NULL REFERENCES bank_account(account_id),
    to_account_id INT NOT NULL REFERENCES bank_account(account_id),
    amount NUMERIC(12,2) NOT NULL CHECK (amount > 0),
```

```

    created_at      TIMESTAMP DEFAULT NOW()
);

```

Code language: SQL (Structured Query Language) (sql)

Note: SELECT SUM(balance) FROM bank_account right after this INSERT returns exactly 1750.00. Every example below relies on that starting total.

Simulating Row-Level Assertions with CHECK

When the invariant only needs data from the row being written, an ordinary CHECK constraint is the simplest and fastest option — already covered in Lab 1. `balance >= 0` on `bank_account`, declared above, is exactly this: it stops any single UPDATE from leaving one account negative, without needing to look at any other row.

Simulating Cross-Row Assertions with Triggers

A CHECK constraint cannot compare a row's new value to its own old value — it only sees NEW. A rule like “no single transaction may reduce an account's balance by more than 60% of what it had” genuinely needs both, so it has to be a trigger:

```

CREATE OR REPLACE FUNCTION guard_large_withdrawal()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.balance < OLD.balance * 0.4 THEN
        RAISE EXCEPTION
            'withdrawal too large for account %: % -> % drops below 40%% of the original
            balance',
            OLD.account_id, OLD.balance, NEW.balance;
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_guard_large_withdrawal
BEFORE UPDATE ON bank_account
FOR EACH ROW
WHEN (NEW.balance < OLD.balance)
EXECUTE FUNCTION guard_large_withdrawal();

```

Code language: SQL (Structured Query Language) (sql)

Deferrable Constraint Triggers

The real assertion — total balance across every account stays at 1750.00 — cannot be checked mid-transfer: a transfer necessarily debits one account before it credits the other, so the sum is briefly wrong between those two statements. A CONSTRAINT TRIGGER marked DEFERRABLE INITIALLY DEFERRED solves exactly this: it still fires once per row change, but PostgreSQL delays actually running it until COMMIT, so only the final state of the transaction is judged:

```

CREATE OR REPLACE FUNCTION check_total_balance_conserved()

```

```

RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_total NUMERIC;
BEGIN
    SELECT SUM(balance) INTO v_total FROM bank_account;

    IF v_total <> 1750.00 THEN
        RAISE EXCEPTION
            'assertion violated: total balance is % but should always be 1750.00', v_total;
    END IF;
    RETURN NULL;
END;
$$;

CREATE CONSTRAINT TRIGGER trg_total_balance_conserved
    AFTER UPDATE ON bank_account
    DEFERRABLE INITIALLY DEFERRED
    FOR EACH ROW
    EXECUTE FUNCTION check_total_balance_conserved();

```

Code language: SQL (Structured Query Language) (sql)

With the constraint trigger in place, a correct transfer procedure passes the assertion because both UPDATES have happened by the time COMMIT runs the check:

```

CREATE OR REPLACE PROCEDURE transfer_funds(
    p_from_account INT,
    p_to_account   INT,
    p_amount       NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE bank_account SET balance = balance - p_amount WHERE account_id = p_from_account;
    UPDATE bank_account SET balance = balance + p_amount WHERE account_id = p_to_account;

    INSERT INTO bank_transaction (from_account_id, to_account_id, amount)
    VALUES (p_from_account, p_to_account, p_amount);

    COMMIT;
END;
$$;

CALL transfer_funds(1, 2, 100.00); -- Alice -> Bob: total stays 1750.00, assertion holds

```

Code language: SQL (Structured Query Language) (sql)

Note: Without DEFERRABLE INITIALLY DEFERRED, the first UPDATE inside transfer_funds would fail the check immediately — the total is momentarily 1650.00 after only the debit has run — even though the procedure was always going to restore it to 1750.00 before COMMIT.

Putting It All Together

A realistic trigger-based safeguard typically combines a lookup, a conditional check, and an exception, exactly like the procedures from Lab 3 — the difference is that a trigger runs automatically, with no CALL required. This BEFORE UPDATE trigger stops a film's rental_rate from ever being lowered by more than 50% in a single change, while still logging every rate change that is allowed through:

```
CREATE TABLE rate_change_log (
  log_id      SERIAL PRIMARY KEY,
  film_id     INT,
  old_rate    NUMERIC,
  new_rate    NUMERIC,
  changed_at  TIMESTAMP DEFAULT NOW()
);

CREATE OR REPLACE FUNCTION guard_and_log_rate_change()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
  IF NEW.rental_rate < OLD.rental_rate * 0.5 THEN
    RAISE EXCEPTION
      'rate cut too large for film %: % -> % is more than a 50%% drop',
      OLD.film_id, OLD.rental_rate, NEW.rental_rate;
  END IF;

  INSERT INTO rate_change_log (film_id, old_rate, new_rate)
  VALUES (NEW.film_id, OLD.rental_rate, NEW.rental_rate);

  RETURN NEW;
END;
$$;

CREATE TRIGGER trg_guard_rate_change
  BEFORE UPDATE ON film
  FOR EACH ROW
  WHEN (OLD.rental_rate IS DISTINCT FROM NEW.rental_rate)
  EXECUTE FUNCTION guard_and_log_rate_change();

UPDATE film SET rental_rate = 0.99 WHERE film_id = 1; -- allowed or rejected depending on the
current rate
```

Code language: SQL (Structured Query Language) (sql)

Exercise

Exercises 1–6 use the dvdrental tables from earlier labs. Exercises 7–8 use the bank_account and bank_transaction tables created in the Assertions section — make sure that schema and its seed data exist before attempting them. Create each trigger function and its CREATE TRIGGER statement, then test it and record the output as your lab report.

1. Write a BEFORE INSERT trigger on payment that rejects any payment with amount ≤ 0 , raising an exception that includes the rejected amount.
2. Write an AFTER UPDATE trigger on film that logs every change to rental_rate into a film_price_history table, storing film_id, old_rate, new_rate, and changed_at.
3. Write a BEFORE DELETE trigger on actor that prevents deleting an actor who still appears in film_actor, raising an exception naming the actor.
4. Write a BEFORE UPDATE trigger on customer that automatically uppercases the email column whenever it is changed, without requiring the caller to do so.
5. Write a trigger-based assertion on rental that prevents a BEFORE INSERT if the chosen inventory_id is already out on an unreturned rental — the same rule the rent_film procedure from Lab 3 checked manually, now enforced automatically for every insert regardless of caller.
6. Write a statement-level AFTER INSERT trigger on rental, using a transition table, that raises a notice reporting how many rows the statement just inserted.
7. Using bank_account and bank_transaction, write a BEFORE UPDATE trigger that rejects any single change that would take an account's balance below 0 — write it as a trigger rather than relying on the existing CHECK constraint, so it can log the rejected attempt into a new insufficient_funds_log table (account_id, attempted_balance, attempted_at) before raising the exception.
8. Using bank_account and bank_transaction, write a DEFERRABLE constraint trigger that asserts every bank_transaction row is fully reflected in the two accounts it references — specifically, that from_account_id's balance plus to_account_id's balance across the whole table always equals the same combined total those two accounts started with. Test it by calling transfer_funds from this section with a transfer between two of the seeded accounts.